torch.autograd.gradcheck.gradcheck#

https://pytorch.org/docs/stable/generated/torch.autograd.gradcheck.gradcheck

Description:

Check gradients computed via small finite differences against analytical gradients wrt tensors in inputs that are of floating point or complex type and with requires_grad=True. The check between numerical and analytical gradients uses allclose(). For most of the complex functions we consider for optimization purposes, no notion of Jacobian exists. Instead, gradcheck verifies if the numerical and analytical values of the Wirtinger and Conjugate Wirtinger derivatives are consistent. Because the gradient computation is done under the assumption that the overall function has a real-valued output, we treat functions with complex output in a special way. For these functions, gradcheck is applied to two real-valued functions corresponding to taking the real components of the complex outputs for the first, and taking the imaginary components of the complex outputs for the second. For more details, check out Autograd for Complex Numbers. The default values are designed for input of double precision. This check will likely fail if input is of less precision, e.g., FloatTensor.

Function Signature

```
torch.autograd.gradcheck.gradcheck(func, inputs, *, eps=1e-
06, atol=1e-05, rtol=0.001, raise_exception=True,
nondet_tol=0.0, check_undefined_grad=True,
check_grad_dtypes=False, check_batched_grad=False,
check_batched_forward_grad=False, check_forward_ad=False,
check_backward_ad=True, fast_mode=False, masked=None)
[source]#

Parameters

Returns

Return type
```

Returns:

bool

Notes & Warnings

NOTE Note The default values are designed for input of double precision. This check will likely fail if input is of less precision, e.g., FloatTensor.

NOTE Note Gradcheck may fail when evaluated on non-differentiable points because the numerically computed gradients via finite differencing may differ those computed analytically (not necessarily because either is incorrect). For more context, see Gradients for non-differentiable functions.

⚠ WARNING Warning If any checked tensor in input has overlapping memory, i.e., different indices pointing to the same memory address (e.g., from torch.Tensor.expand()), this check will likely fail because the numerical gradients computed by point perturbation at such indices will change values at all other indices that share the same memory address.

Detailed Documentation

Documentation

Check gradients computed via small finite differences against analytical gradients wrt tensors in inputs that are of floating point or complex type and with requires_grad=True.

The check between numerical and analytical gradients uses allclose().

For most of the complex functions we consider for optimization purposes, no notion of Jacobian exists. Instead, gradcheck verifies if the numerical and analytical values of the Wirtinger and Conjugate Wirtinger derivatives are consistent. Because the gradient computation is done under the assumption that the overall function has a real-valued output, we treat functions with complex output in a special way. For these functions, gradcheck is applied to two real-valued functions corresponding to taking the real components of the complex outputs for the first, and taking the imaginary components of the complex outputs for the second. For more details, check out Autograd for Complex Numbers.

The default values are designed for input of double precision. This check will likely fail if input is of less precision, e.g., FloatTensor.

Gradcheck may fail when evaluated on non-differentiable points because the numerically computed gradients via finite differencing may differ those computed analytically (not necessarily because either is incorrect). For more context, see Gradients for non-differentiable functions.

If any checked tensor in input has overlapping memory, i.e., different indices pointing to the same memory address (e.g., from torch.Tensor.expand()), this check will likely fail because the numerical gradients computed by point perturbation at such indices will change values at all other indices that share the same memory address.

func (function) – a Python function that takes Tensor inputs and returns a Tensor or a tuple of Tensors

inputs (tuple of Tensor or Tensor) – inputs to the function

eps (float, optional) – perturbation for finite differences

atol (float, optional) – absolute tolerance

rtol (float, optional) – relative tolerance

raise_exception (bool, optional) – indicating whether to raise an exception if the check fails. The exception gives more information about the exact nature of the failure. This is helpful when debugging gradchecks.

nondet_tol (float, optional) – tolerance for non-determinism. When running identical inputs through the differentiation, the results must either match exactly (default, 0.0) or be within this tolerance.

check_undefined_grad (bool, optional) – if True, check if undefined output grads are supported and treated as zeros, for Tensor outputs.

check_batched_grad (bool, optional) – if True, check if we can compute batched gradients using prototype vmap support. Defaults to False.

check_batched_forward_grad (bool, optional) – if True, checks if we can compute batched forward gradients using forward ad and prototype vmap support. Defaults to False.

check_forward_ad (bool, optional) – if True, check that the gradients computed with forward mode AD match the numerical ones. Defaults to False.

check_backward_ad (bool, optional) – if False, do not perform any checks that rely on backward mode AD to be implemented. Defaults to True.

fast_mode (bool, optional) – Fast mode for gradcheck and gradgradcheck is currently only implemented for R to R functions. If none of the inputs and outputs are complex a faster implementation of gradcheck that no longer computes the entire jacobian is run; otherwise, we fall back to the slow implementation.

masked (bool, optional) – if True, the gradients of unspecified elements of sparse tensors are ignored. Defaults to False.

True if all differences satisfy allclose condition
